Modeling Report: Imbalance Handling and Gradient Boosting

Priyanshu, Imbalance and Modeling Lead. Explainable Credit Risk Assessment (Give Me Some Credit).
Draft: baseline complete, 12-experiment results to be added.

1. Objective

This report documents the modeling stage of the project. It compares four imbalance-handling techniques (none, class weights, SMOTE, undersampling) across three models (Logistic Regression, XGBoost, LightGBM), and prepares the best model for tuning and hand-off to Member 3 for evaluation and SHAP analysis.

2. Data

Input comes from Member 1's cleaned, leak-free split of the Give Me Some Credit dataset.

Item	Value
Training rows	119,992
Test rows	29,994
Features	10
Target	SeriousDlqin2yrs (1 = default)
Default rate	about 6.68% in both sets
Missing values	none after median imputation

Preprocessing (done by Member 1): invalid ages removed, late-payment counts capped at 20, utilization, debt ratio and income capped at the 99th percentile of train, and medians computed on the training set only. The test set is reserved for final evaluation.

Features

Feature	Description	Type
RevolvingUtilizationOfUnsecuredLines	Share of credit limit used	Ratio
age	Borrower age	Integer
NumberOfTime30-59DaysPastDueNotWorse	Times 30-59 days late	Count
DebtRatio	Monthly debt payments / monthly income	Ratio
MonthlyIncome	Monthly income	Continuous
NumberOfOpenCreditLinesAndLoans	Open loans and credit lines	Count
NumberOfTimes90DaysLate	Times 90+ days late	Count
NumberRealEstateLoansOrLines	Real estate loans	Count
NumberOfTime60-89DaysPastDueNotWorse	Times 60-89 days late	Count
NumberOfDependents	Number of dependents	Count

The target SeriousDlqin2yrs is 1 when the borrower was 90 or more days late within two years. In the modeling code, hyphens in feature names are replaced with underscores because LightGBM rejects them.

How imbalanced the data is

In the original 150,000 borrowers, 139,974 (93.32%) did not default and 10,026 (6.68%) did, a ratio of about 13.96 to 1. After cleaning and the stratified split, the training set has about 8,016 defaulters among 119,992 rows. A model that predicts "no default" for everyone would be about 93% accurate and catch 0%

of defaulters, which is why plain accuracy is not used in this project.

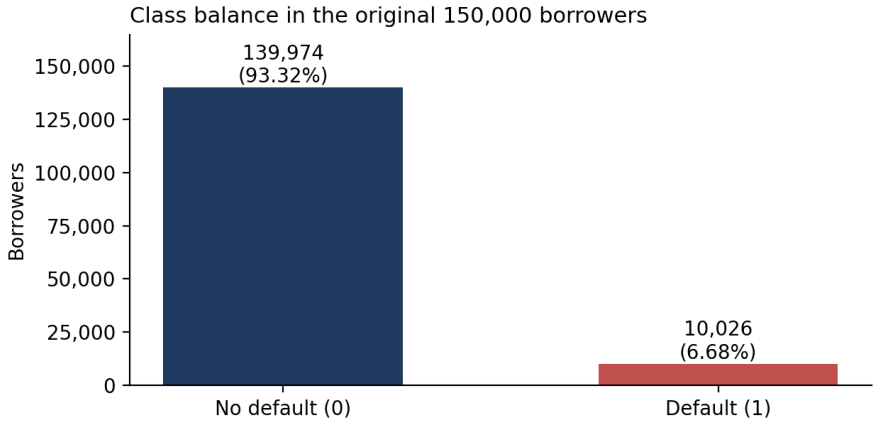


Figure 1. *Class balance of the original dataset (counts from Member 1's inspection). Only about 1 borrower in 15 defaults.*

Leakage safeguards

- The split is stratified 80/20 with random_state=42, done before any statistic is computed.
- Medians for imputation and 99th-percentile caps come from the training set only and are applied to both sets.
- Resampling (SMOTE, undersampling) happens only inside training folds during modeling.
- test.csv is not loaded by the experiment script and is reserved for final evaluation.

3. Experimental Design

Models. Logistic Regression (baseline, with standard scaling), XGBoost and LightGBM. Both boosters use fixed, identical-style settings (300 trees, learning rate 0.05, subsampling 0.8) so that differences come from the imbalance technique rather than hyperparameters. Tuning is done afterwards on the best model only.

Imbalance techniques.

Technique	Implementation
None	Train on the data as is
Class weight	class_weight=balanced (LogReg, LightGBM); scale_pos_weight of about 13.96 (XGBoost)
SMOTE	Synthetic minority samples, inside training folds only
Undersampling	Random removal of majority-class rows, inside training folds only

Validation. 5-fold stratified cross-validation with random_state=42 on the training set. Resampling is applied through an imblearn Pipeline so validation folds are never resampled and no synthetic points leak into them.

Metrics. PR-AUC (primary, because only about 7% of borrowers default), ROC-AUC, and recall at the default 0.5 threshold. Plain accuracy is not used because it is misleading on imbalanced data. Threshold tuning is left to Member 3.

Why these metrics

- **PR-AUC** is the primary metric. It focuses on the minority class, and a random model scores about 0.067 (the default rate), so the baseline's 0.313 is roughly 4.7 times chance.
- **ROC-AUC** measures how well the model ranks defaulters above non-defaulters at any threshold. It can look optimistic when non-defaulters dominate, so it is a secondary check.

- **Recall** is the share of real defaulters the model catches. A missed defaulter is the costly error for a lender. Precision, F1, KS, calibration and threshold tuning belong to Member 3.

Why 5-fold stratified cross-validation

Cross-validation uses every training row for both fitting and validation, which gives a mean and a spread instead of one lucky split. Stratification keeps the default rate near 6.68% in every fold (about 1,600 defaulters per validation fold), and shuffling with random_state=42 makes the folds repeatable.

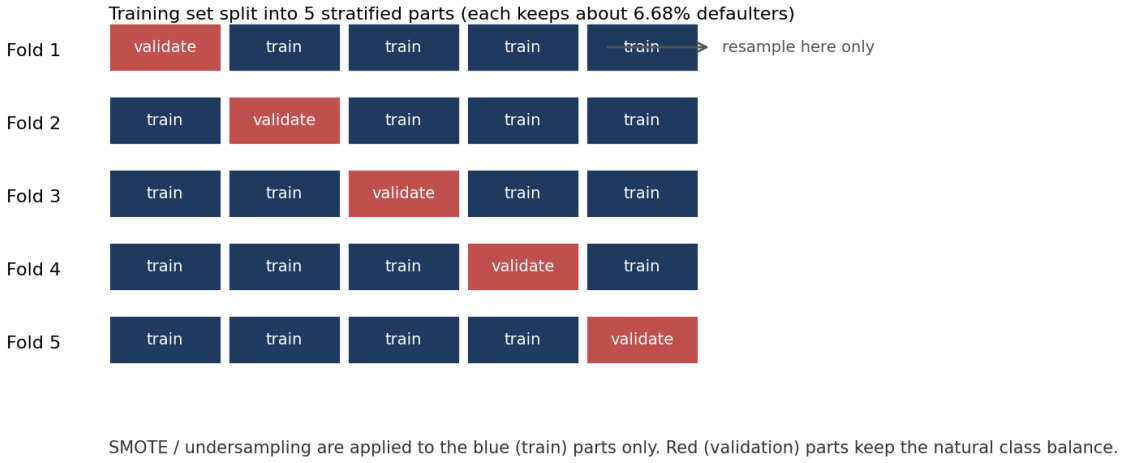


Figure 2. *How 5-fold stratified cross-validation is used. Resampling touches only the training parts, so validation scores reflect the real 6.68% default rate.*

Pipeline for each experiment

Model	Step order
Logistic Regression	StandardScaler, then sampler (if SMOTE or undersampling), then model
XGBoost	Sampler (if SMOTE or undersampling), then model
LightGBM	Sampler (if SMOTE or undersampling), then model

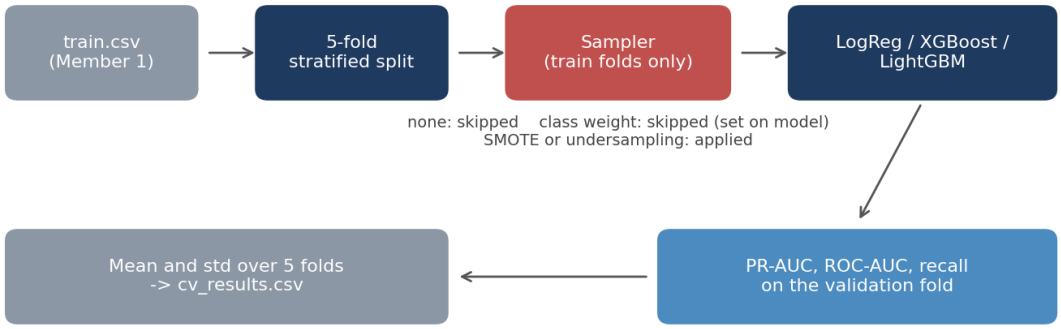


Figure 3. *Flow of one experiment. The sampler sits inside the imblearn Pipeline, so it runs only when the pipeline is fit on a training fold.*

Trees do not need scaling. The scaler is fit inside each training fold so validation data never influences it.

Model settings

Model	Fixed settings
Logistic Regression	max_iter=1000; class_weight=balanced for the class-weight technique

Model	Fixed settings
XGBoost	300 trees, learning rate 0.05, max_depth 4, subsample 0.8, colsample_bytree 0.8, hist tree method, aucpr metric; scale_pos_weight about 13.96 for the class-weight technique, 1 otherwise
LightGBM	300 trees, learning rate 0.05, num_leaves 31, subsample 0.8, colsample_bytree 0.8; class_weight=balanced for the class-weight technique

How each technique changes training

- **None:** the model sees the natural 6.68% default rate.
- **Class weight:** each defaulter counts about 14 times more in the loss, so errors on defaulters cost more. No rows are added or removed.
- **SMOTE:** creates synthetic defaulters by interpolating between a real defaulter and its nearest minority neighbours until both classes match. Each training fold grows from about 96,000 to about 179,000 rows.
- **Undersampling:** randomly drops non-defaulters until both classes match. Each training fold shrinks to about 12,800 rows, which is fast but discards information.

Reproducibility

random_state=42 is used for the folds, samplers and models. The script saves cv_results.csv after every experiment, skips finished ones on rerun, and can run one model at a time with --models, so the work can be split across two computers.

4. Baseline Results (Logistic Regression, no imbalance handling)

Metric	CV mean	CV std	Test (one-time check)
PR-AUC	0.313	0.011	0.327
ROC-AUC	0.821	0.004	0.830
Recall (0.5)	0.063	0.004	0.067

The baseline separates borrowers reasonably well (ROC-AUC above 0.82), but at the default threshold it catches only about 6% of defaulters. This gap between ranking quality and recall is the main motivation for testing imbalance techniques.

The test column is a single sanity check of a train-fitted baseline. It confirms there is no train/test mismatch, since test scores sit within the CV spread. It was not used for any modeling decision, and all model selection stays in cross-validation.

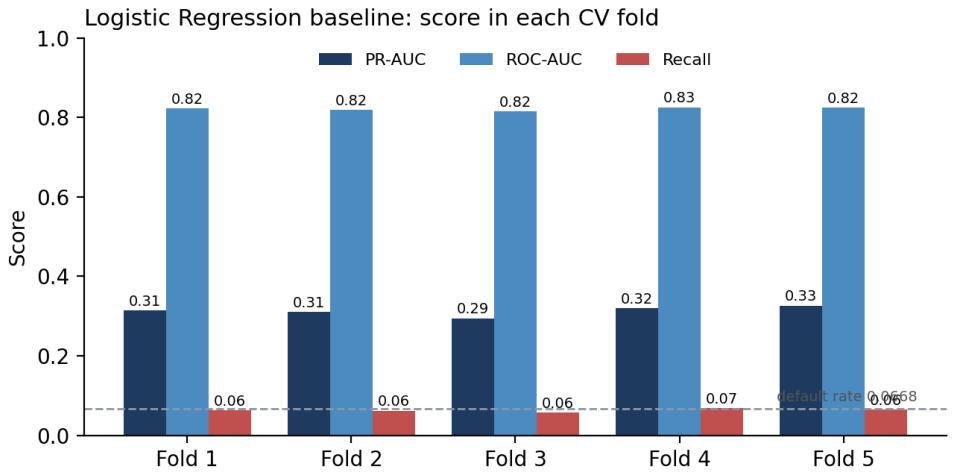


Figure 4. *Per-fold CV scores of the Logistic Regression baseline (real output of baseline_logreg.py). Recall sits near the dashed line of the default rate.*

Reading the baseline

- **ROC-AUC 0.821:** a randomly chosen defaulter gets a higher risk score than a randomly chosen non-defaulter about 82% of the time.
- **PR-AUC 0.313:** about 4.7 times the 0.0668 default rate, so the model finds real signal, but precision falls quickly as recall rises.
- **Recall 0.063:** at a 0.5 cutoff the model flags only borrowers whose estimated default probability is at least 50%. With a 6.7% base rate, very few borrowers reach that level, so most defaulters are missed.
- **Stability:** the small CV standard deviations (0.011 for PR-AUC, 0.004 for ROC-AUC) show the estimate does not depend heavily on the fold split.

Expectation to check, not a result: class weights, SMOTE and undersampling push predicted probabilities upward, so recall should rise while precision falls. PR-AUC may change little, because these techniques mostly shift scores rather than improve ranking.

5. Results of the 12 Experiments

To be filled from cv_results.csv after running run_experiments.py.

Model	Technique	PR-AUC	ROC-AUC	Recall
LogReg	none	0.313	0.821	0.063
LogReg	class_weight	...	...	...
LogReg	smote	...	...	...
LogReg	undersample	...	...	...
XGBoost	none	...	...	...
XGBoost	class_weight	...	...	...
XGBoost	smote	...	...	...
XGBoost	undersample	...	...	...
LightGBM	none	...	...	...
LightGBM	class_weight	...	...	...
LightGBM	smote	...	...	...
LightGBM	undersample	...	...	...

Interpretation to write once results are in: which technique raises recall, what it costs in PR-AUC, and how much gradient boosting improves on the baseline.

6. Hyperparameter Tuning Plan

The best model by mean PR-AUC will be tuned with Optuna (30 to 50 trials, objective: mean 5-fold CV PR-AUC). Search space: learning rate, number of estimators, tree depth or leaves, subsampling, column sampling and minimum child weight.

Proposed search space

Parameter	Range	Scale
learning_rate	0.01 to 0.3	log
n_estimators	100 to 800	integer
max_depth (XGBoost) or num_leaves (LightGBM)	3 to 10, or 15 to 127	integer
subsample	0.6 to 1.0	linear
colsample_bytree	0.6 to 1.0	linear
min_child_weight	1 to 10	integer
reg_lambda	0.001 to 10	log

These ranges are a starting proposal and can be adjusted before the run.

Tuning procedure

1. Pick the best model and technique by mean PR-AUC in the 12-experiment table, checking that the gap to the runner-up is larger than the CV standard deviation.
2. Run Optuna with a fixed seed, each trial scoring mean PR-AUC over the same 5 stratified folds, using the winning technique inside the imblearn pipeline.
3. Refit the best parameters on the full training set.
4. Refit the same tuned parameters under all four techniques so Member 3 can compare SHAP across them, and save each with joblib.

7. Hand-off to Member 3

- Final models saved with joblib in models/. Because the SHAP stability analysis compares the same model across all four techniques, all four variants of the best model are saved, not only the winner.
- cv_results.csv with mean and standard deviation of each metric.
- Methodology notes (this report) and the random_state=42 convention.
- Note on feature names: hyphens were replaced with underscores for LightGBM compatibility.

8. Limitations

- Boosters use fixed settings during the comparison, so a technique could rank differently after individual tuning.
- Recall is measured at a 0.5 threshold, which understates what each model can achieve with a tuned threshold.
- SMOTE interpolates in a feature space that includes counts and capped ratios, so some synthetic points may not be realistic borrowers.

9. Viva Preparation

Question	Short answer
What is SMOTE, and why only on training folds?	SMOTE creates synthetic defaulters by interpolating between neighbouring real ones. If applied before splitting, synthetic points built from validation rows leak into training and inflate scores, so it runs only on training folds.
Class weight vs SMOTE?	Class weights change the loss and add no data, so they are fast and keep the data real. SMOTE changes the data itself, which is slower and can create unrealistic borrowers.
Why PR-AUC and not accuracy?	With 6.68% defaulters, predicting no default for everyone gives about 93% accuracy. PR-AUC measures how well the minority class is found.
XGBoost vs LightGBM?	Both are gradient boosting. XGBoost grows trees level by level, while LightGBM grows the leaf with the largest gain, which is usually faster on large data but can overfit without limits such as num_leaves.
Why cross-validation?	It uses all training data for fitting and validation, gives a mean and a spread, and keeps the test set untouched for the final check.
Why is baseline recall so low?	At a 0.5 threshold few borrowers reach a 50% default probability when the base rate is 6.7%. The model ranks well (ROC-AUC 0.82) but is cautious about flagging.
Why was the test set not used for tuning?	Choosing models on test data would make the final scores optimistic. All selection happens in cross-validation.

Everyone should understand the full pipeline, so also be ready to explain Member 1's imputation and capping, and Member 3's SHAP analysis.

10. Remaining Work

- [ ] Install imbalanced-learn, xgboost and lightgbm on the modeling machine
- [ ] Run run_experiments.py and produce cv_results.csv
- [ ] Fill the results table in section 5 and write the interpretation
- [ ] Choose the best model and technique, then tune with Optuna
- [ ] Refit and save all four technique variants with joblib
- [ ] Hand off models and notes to Member 3
- [ ] Open a pull request on the member2-modeling branch

Appendix A. Working in VS Code

The figures below show the project in Visual Studio Code. They are rendered illustrations in the VS Code dark theme, built from the real project files and the real output of baseline_logreg.py run on train.csv. They are not raw screen captures, so window details on your machine (theme, fonts, shell prompt) will differ slightly.

A.1 Open the project and the baseline script

1 In VS Code choose File, then Open Folder, and select your repository folder.

2 The Explorer on the left should show data/train.csv, data/test.csv, baseline_logreg.py and run_experiments.py.

3 Click baseline_logreg.py to open it. The green comments beginning with # %% split the file into cells.

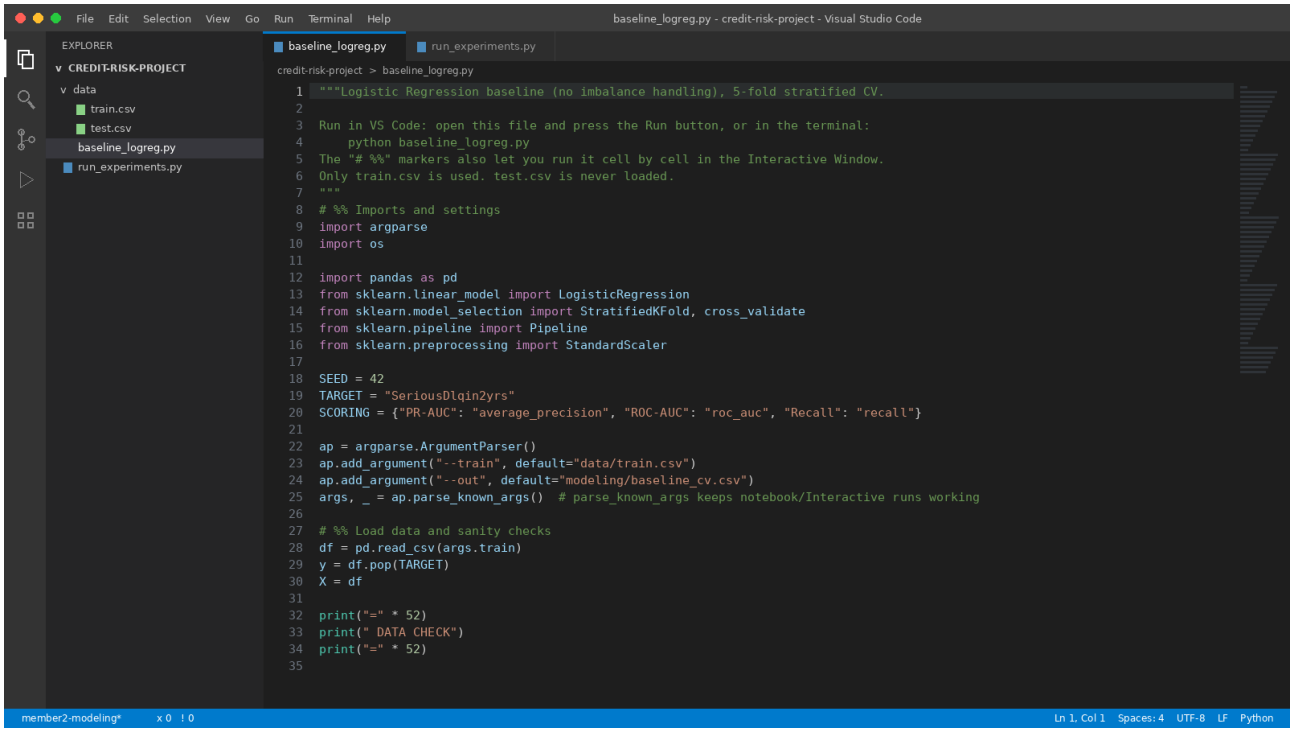


Figure 5. *Project open in VS Code with baseline_logreg.py in the editor.*

A.2 Run the baseline and read the output

1 Open the terminal with Terminal, then New Terminal (shortcut Ctrl+backtick).

2 Type python baseline_logreg.py and press Enter.

3 The output prints a data check, the per-fold scores and the summary, then saves modeling/baseline_cv.csv.

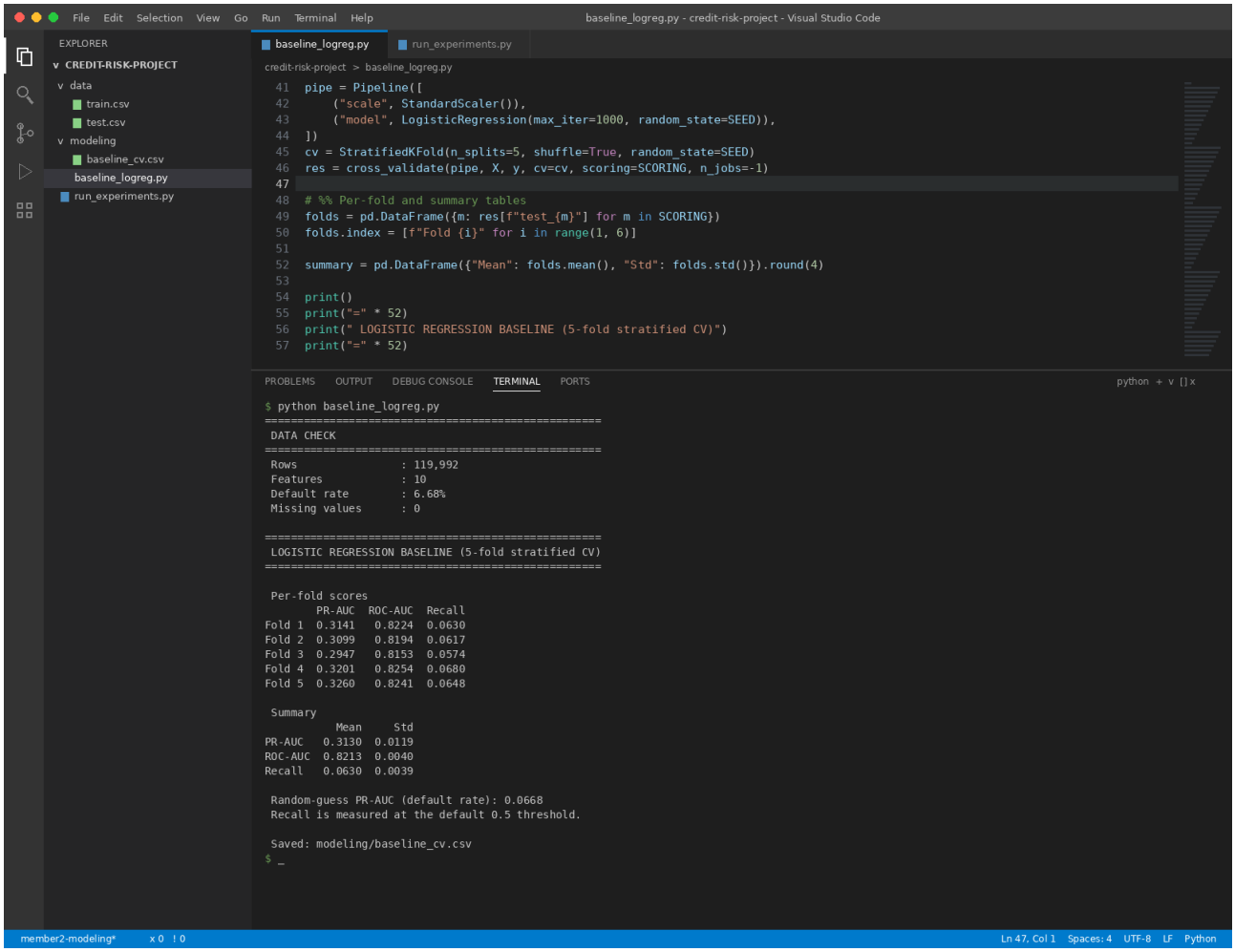


Figure 6. *Integrated terminal showing the real output of the baseline run: 119,992 rows, 10 features, 6.68% defaults and 0 missing values, followed by the per-fold and summary tables.*

A.3 The saved results file

The summary is saved to modeling/baseline_cv.csv. Open it from the Explorer to confirm it holds the three metrics with their mean and standard deviation.

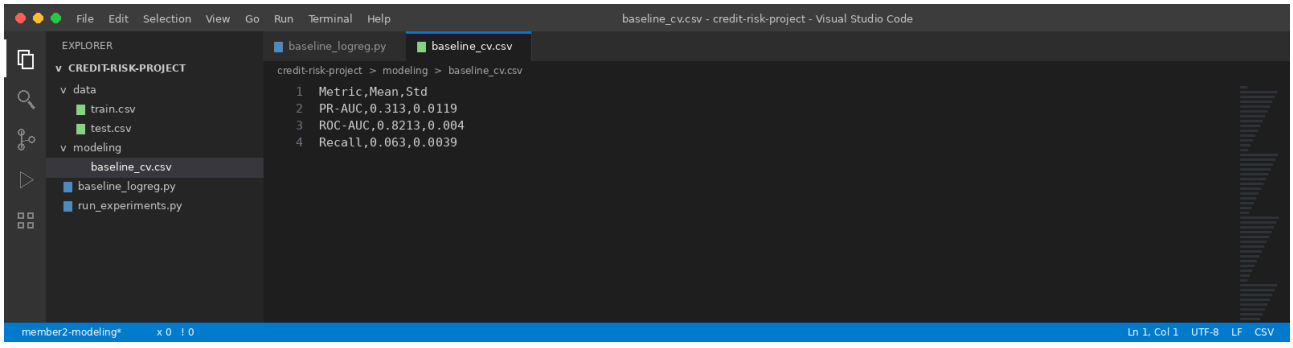


Figure 7. *modeling/baseline_cv.csv opened in the editor (real file contents).*

A.4 Running the 12 experiments

run_experiments.py runs all 3 models with all 4 techniques. Install the packages first, then run the whole set, or one model at a time if the work is split across two computers.

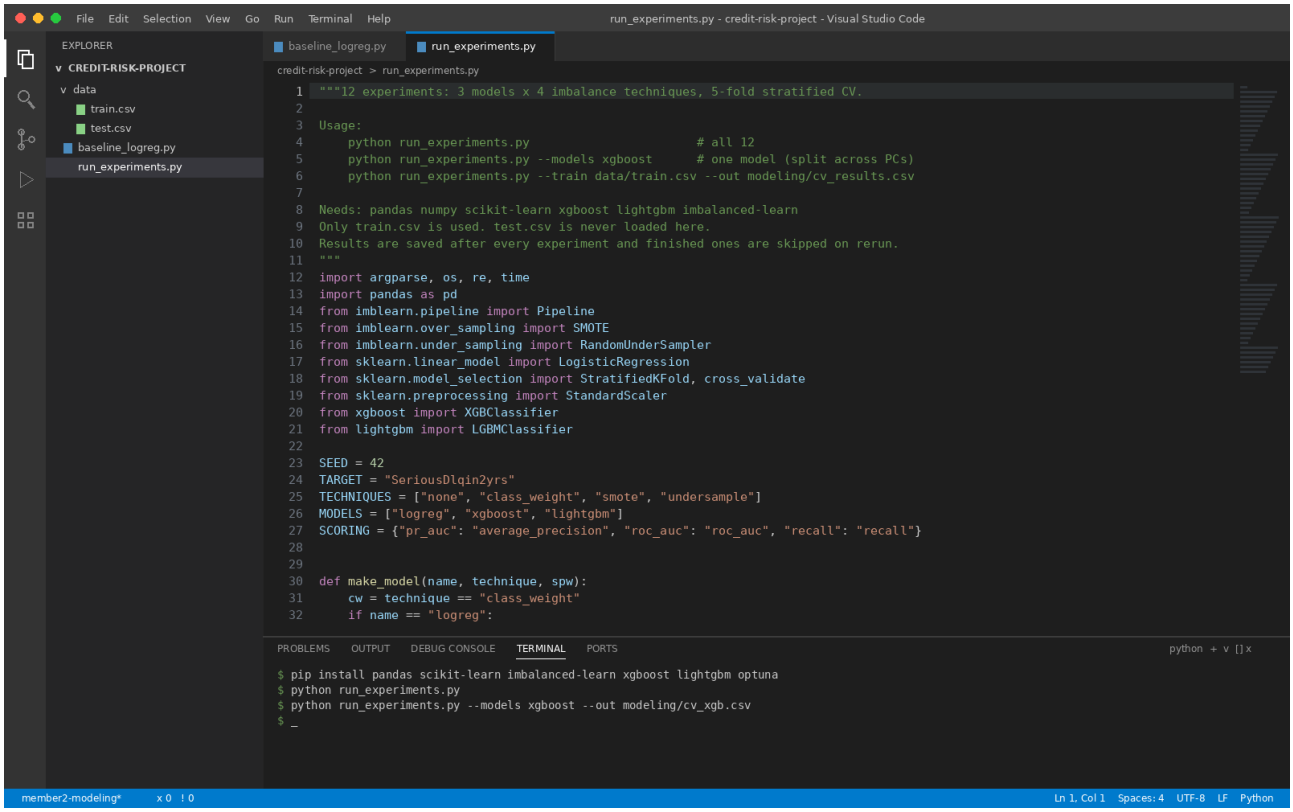


Figure 8. *run_experiments.py in the editor with the commands to run in the terminal. The run has not been executed yet, so no results are shown; the output will be added to section 5 afterwards.*

A.5 Common problems

Problem	Fix
ModuleNotFoundError: imblearn	Run pip install imbalanced-learn in the terminal.
FileNotFoundError: data/train.csv	Open the repository root folder in VS Code, or pass --train path/to/train.csv.
Wrong Python interpreter	Press Ctrl+Shift+P, choose Python: Select Interpreter, and pick the environment where you installed the packages.
Run button runs the wrong file	Click the tab of the file you want first, then press Run.